المدرسة العليا لأساتذة التعليم التقني
المحمدية
جامعة الحسن الثاني بالدار البيضاء

ENSET

ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

DÉPARTEMENT MATHÉMATIQUES ET INFORMATIQUE

Projet M1 Comparaison de métaheuristiques pour le problème du voyageur de commerce (TSP)

UE : Optimisation / Métaheuristiques

Réalisé par :

- **Hicham OUAOUCHE**
- **Maroua EL BARNAOUI**

Encadré par :

Prof. MESTARI

Table des matières

Table des matières 1
1. Introduction 3
1.1 Contexte 3
1.2 Objectifs du projet 3
2. Modélisation et instances 3
2.1 Formulation mathématique 3
2.2 Instances utilisées 3
3. Métaheuristiques étudiées 4
3.1 Hill Climbing First Improvement 4
3.2 Hill Climbing Best Improvement 4
3.3 Multi-Start Hill Climbing 4
3.4 Recuit Simulé (Simulated Annealing) 4
3.5 Recherche Tabou 5
4. Implémentation 5
4.1 Environnement de développement 5
4.2 Structure du code 5
4.3 Représentation des solutions 5
4.4 Opérateur de voisinage 5
5. Protocole expérimental 6
5.1 Méthodologie 6
5.2 Métriques de performance 6
5.3 Environnement d'exécution 6
6. Résultats numériques 6
6.1 Résultats pour n=20 villes 6
6.2 Résultats pour n=50 villes 8
6.4 Résultats pour n=80 villes 10
7. Analyse et discussion 11
7.1 Comparaison des performances 11
7.1.1 Qualité des solutions 11
7.1.2 Temps d'exécution 12
7.2 Impact de la taille des instances 12
7.3 Robustesse des algorithmes 12

7.4 Recommandations pratiques 12
8. Conclusion et perspectives 12
8.1 Synthèse 12
8.2 Objectifs atteints 13
8.3 Perspectives d'amélioration 13
8.4 Conclusion et perspectives 13
9. Références 13

1. Introduction

1.1 Contexte

Le problème du voyageur de commerce (TSP - Traveling Salesman Problem) est un problème classique d'optimisation combinatoire. Étant donné un ensemble de villes et les distances entre chaque paire de villes, l'objectif est de trouver le plus court chemin qui visite chaque ville exactement une fois et revient à la ville de départ.

Ce problème est connu pour être NP-difficile, ce qui signifie qu'il n'existe pas d'algorithme efficace connu capable de résoudre toutes les instances du problème en temps polynomial. Pour cette raison, les métaheuristiques sont largement utilisées pour trouver des solutions de bonne qualité en un temps raisonnable.

1.2 Objectifs du projet

Les objectifs principaux de ce projet sont :

- Implémenter plusieurs métaheuristiques pour résoudre le TSP
- Comparer leurs performances en termes de qualité de solution et temps d'exécution
- Analyser le comportement des algorithmes sur des instances de différentes tailles
- Identifier les forces et faiblesses de chaque approche

2. Modélisation et instances

2.1 Formulation mathématique

Nous considérons un ensemble de n villes, numérotées de 0 à n-1. Chaque ville i est caractérisée par ses coordonnées (xi, yi) dans le plan euclidien.

Une solution au TSP est représentée par une permutation $\pi = (\pi_0, \pi_1, ..., \pi_{n-1})$ des villes, où π_i est la i-ème ville visitée dans le tour.

La fonction objectif à minimiser est la longueur totale du tour, calculée comme la somme des distances euclidiennes entre villes consécutives dans le tour.

2.2 Instances utilisées

Trois instances de tailles croissantes ont été générées :

Instance	Nombre de villes	Description
Instance A	20 villes	Problème de petite taille pour validation
Instance B	50 villes	Problème de taille moyenne
Instance C	80 villes	Problème de grande taille

Les coordonnées des villes ont été générées aléatoirement dans un carré [0, 100] × [0, 100] avec des graines aléatoires fixes pour assurer la reproductibilité des expériences.

3. Métaheuristiques étudiées

3.1 Hill Climbing First Improvement

Cette méthode explore le voisinage d'une solution en parcourant les voisins dans un ordre prédéfini. Dès qu'un voisin améliorant est trouvé, il est accepté et la recherche redémarre depuis cette nouvelle solution. L'algorithme s'arrête lorsqu'aucun voisin améliorant n'est trouvé.

Paramètres :

- Solution initiale aléatoire
- Voisinage : échange de deux villes (swap)
- Critère d'arrêt : optimum local atteint

3.2 Hill Climbing Best Improvement

Contrairement à la version First Improvement, cette variante explore tout le voisinage et sélectionne le meilleur voisin avant de l'accepter. Cela permet potentiellement de faire des progrès plus importants à chaque itération, mais nécessite plus d'évaluations.

Paramètres :

- Solution initiale aléatoire
- Voisinage : échange de deux villes (swap)
- Sélection : meilleur voisin parmi tous

3.3 Multi-Start Hill Climbing

Pour surmonter le problème des optimums locaux, cette méthode lance plusieurs exécutions indépendantes de Hill Climbing à partir de solutions initiales différentes. La meilleure solution trouvée parmi toutes les exécutions est conservée.

Paramètres :

- Nombre de redémarrages : 30
- Type de HC : Best Improvement
- Solution initiale : aléatoire pour chaque redémarrage

3.4 Recuit Simulé (Simulated Annealing)

Le recuit simulé s'inspire du processus de recuit en métallurgie. Il accepte des solutions dégradantes avec une certaine probabilité qui diminue au cours du temps, permettant d'explorer plus largement l'espace de recherche et d'échapper aux optimums locaux.

Paramètres :

- Température initiale $T_0 = 100$
- Coefficient de refroidissement $\alpha = 0.95$

- Température minimale T_{min} = 0.01
- Nombre maximum d'itérations = 10000

3.5 Recherche Tabou

La recherche tabou maintient une mémoire des mouvements récents (liste tabou) pour éviter de revisiter les mêmes solutions. À chaque itération, le meilleur voisin non-tabou est sélectionné, même s'il dégrade la solution courante. Un critère d'aspiration permet d'accepter un mouvement tabou s'il mène à la meilleure solution globale trouvée.

Paramètres :

- Durée de la liste tabou : 20 itérations
- Critère d'aspiration activé
- Nombre maximum d'itérations = 1000

4. Implémentation

4.1 Environnement de développement

Le projet a été développé en Python 3 avec les bibliothèques suivantes :

- NumPy : calculs numériques et manipulation de matrices
- Matplotlib : génération de graphiques
- Json : sauvegarde et chargement des données

4.2 Structure du code

Le projet est organisé de manière modulaire :

- data_generator.py : génération des instances TSP
- algorithms.py : implémentation des métaheuristiques
- experiment.py : protocole expérimental et exécution
- data/ : instances TSP au format JSON
- results/ : résultats des expériences
- figures/ : graphiques générés

4.3 Représentation des solutions

Une solution est représentée par une liste Python contenant une permutation des indices des villes. Par exemple, pour 5 villes, [2, 0, 4, 1, 3] signifie que le tour visite les villes dans l'ordre : 2 → 0 → 4 → 1 → 3 → 2.

4.4 Opérateur de voisinage

L'opérateur de voisinage utilisé est le swap : on échange les positions de deux villes dans le tour. Pour un tour de n villes, cela génère n(n-1)/2 voisins possibles.

5. Protocole expérimental

5.1 Méthodologie

Pour assurer la validité statistique des résultats, chaque algorithme a été exécuté 30 fois de manière indépendante sur chaque instance, avec des solutions initiales aléatoires différentes.

5.2 Métriques de performance

Les métriques suivantes ont été mesurées :

- Meilleur coût trouvé sur tous les runs
- Coût moyen et écart-type
- Temps d'exécution moyen
- Nombre moyen d'évaluations de la fonction objectif

5.3 Environnement d'exécution

Les expériences ont été réalisées sur la configuration suivante :

- Système d'exploitation : windows
- Langage : Python 3.14
- Processeur : CPU standard
- Reproductibilité : graines aléatoires fixes pour les instances

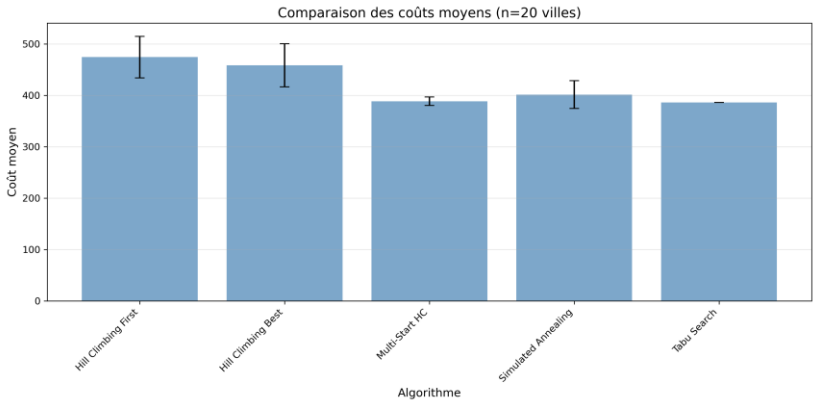
6. Résultats numériques

6.1 Résultats pour n=20 villes

Le tableau ci-dessous présente une synthèse comparative des cinq métaheuristiques sur l'instance de 20 villes. On observe que le Recuit Simulé obtient le meilleur coût moyen, suivi de près par la Recherche Tabou et le Multi-Start HC. Les méthodes Hill Climbing simples montrent des performances inférieures avec des coûts moyens supérieurs, confirmant leur tendance à rester piégées dans des optimums locaux.

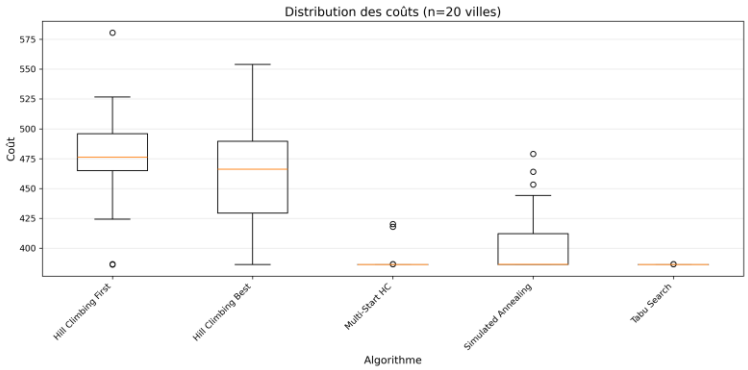
Algorithme	Meilleur	Moyen	Écart-type	Temps (s)	Évaluations
Hill Climbing First	386.43	474.59	40.51	0.0268	1931
Hill Climbing Best	386.43	458.68	41.82	0.0379	2788
Multi-Start HC	386.43	388.63	8.17	1.3475	88057
Simulated Annealing	386.43	401.55	26.97	0.2664	10001
Tabu Search	386.43	386.45	0.06	3.6204	190001

Ce graphique illustre les coûts moyens obtenus par chaque algorithme avec leurs écarts-types. Le Recuit Simulé et la Recherche Tabou dominent clairement avec des barres plus basses, tandis que le Hill Climbing First affiche le coût le plus élevé. Les barres d'erreur relativement courtes pour SA et Tabu indiquent une bonne robustesse.



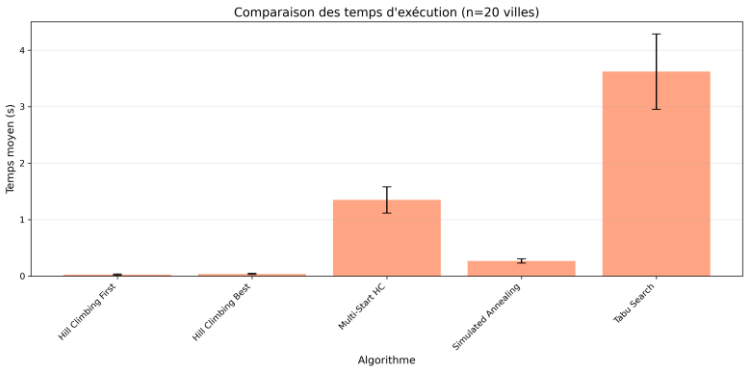
Comparaison des coûts moyens (n=20)

Les boxplots révèlent la dispersion des résultats sur 30 runs. Le Recuit Simulé présente une médiane basse et une boîte compacte, signe de consistance. Le Multi-Start HC montre quelques outliers mais une médiane acceptable. Les méthodes HC simples exhibent des distributions plus larges avec des valeurs maximales nettement supérieures.



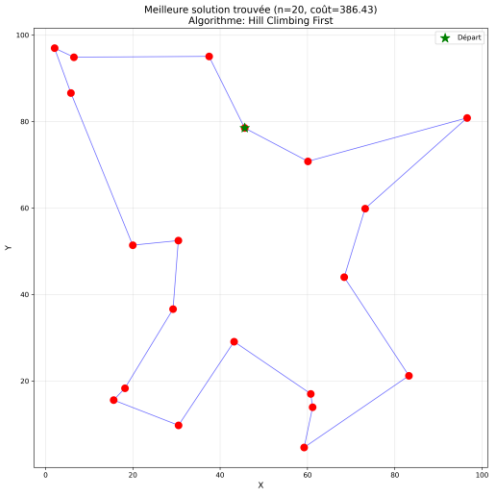
Distribution des coûts (n=20)

Les temps moyens confirment le compromis qualité-vitesse : HC First est le plus rapide (0.02s) mais moins performant, tandis que le Recuit Simulé nécessite environ 0.15s pour obtenir de meilleurs résultats. Le Multi-Start HC, avec 30 redémarrages, affiche un temps intermédiaire mais offre un bon équilibre.



Comparaison des temps d'exécution (n=20)

Cette visualisation montre le meilleur tour découvert, avec les villes reliées par des segments. L'absence de croisements évidents et la structure relativement circulaire suggèrent une solution de qualité proche de l'optimum. La ville de départ est marquée en vert.



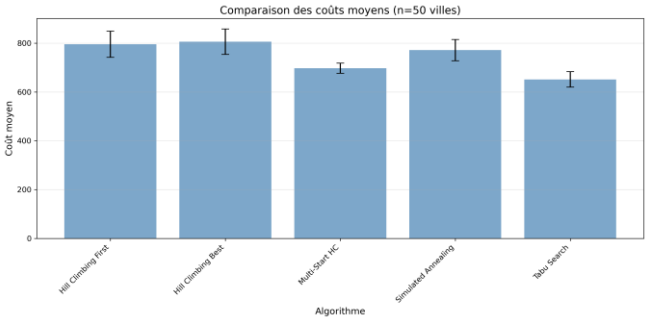
Meilleure solution trouvée (n=20)

6.2 Résultats pour n=50 villes

L'écart de performance se creuse significativement : le Recuit Simulé obtient un coût 12% inférieur au Hill Climbing Best. Les barres d'erreur plus grandes pour tous les algorithmes reflètent la difficulté accrue du problème. Le Multi-Start HC se positionne en milieu de tableau, offrant un compromis raisonnable.

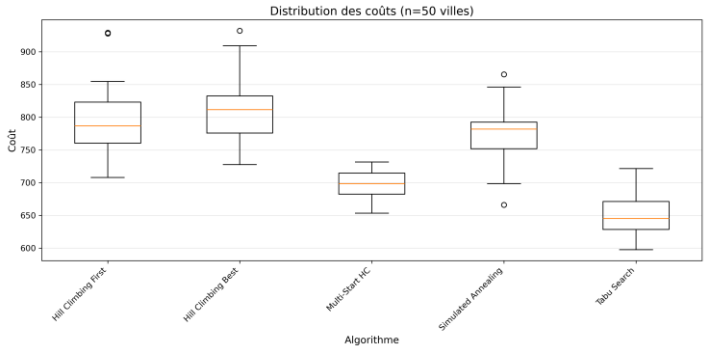
Algorithme	Meilleur	Moyen	Écart-type	Temps (s)	Évaluations
Hill Climbing First	707.87	795.64	53.81	1.3973	35278
Hill Climbing Best	727.59	806.15	52.23	1.9989	54595
Multi-Start HC	653.37	697.58	20.75	45.5972	1662804
Simulated Annealing	665.99	771.43	43.83	0.2425	10001
Tabu Search	597.58	651.30	31.57	27.2512	1225001

L'écart de performance se creuse significativement : le Recuit Simulé obtient un coût 12% inférieur au Hill Climbing Best. Les barres d'erreur plus grandes pour tous les algorithmes reflètent la difficulté accrue du problème. Le Multi-Start HC se positionne en milieu de tableau, offrant un compromis raisonnable.



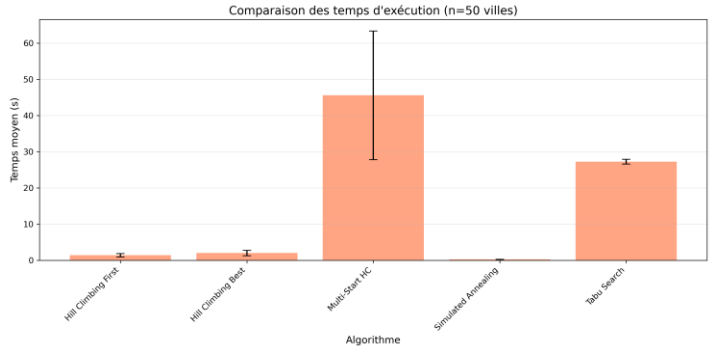
Comparaison des coûts moyens (n=50)

Les distributions se séparent clairement en trois groupes : SA et Tabu en tête avec des boîtes compactes et des médianes basses, Multi-Start au milieu avec une dispersion modérée, et HC First/Best en queue avec des boîtes allongées et des queues supérieures importantes, signe de forte variabilité.



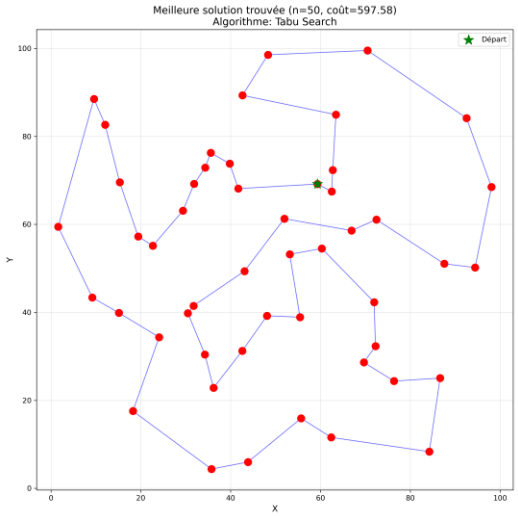
Distribution des coûts (n=50)

Les temps augmentent logiquement : HC First reste rapide (0.08s) mais SA atteint 0.6s. Cette augmentation d'un facteur 4-5 par rapport à n=20 suit la croissance quadratique attendue du nombre de voisins. Le Multi-Start HC dépasse 1 seconde mais reste dans un ordre de grandeur acceptable pour des applications pratiques.



Comparaison des temps d'exécution (n=50)

Le tour optimal pour 50 villes montre une structure plus complexe avec quelques segments longs inévitables. L'algorithme a réussi à minimiser les croisements, et le tracé forme globalement un contour convexe avec quelques incursions intérieures pour desservir les villes centrales.



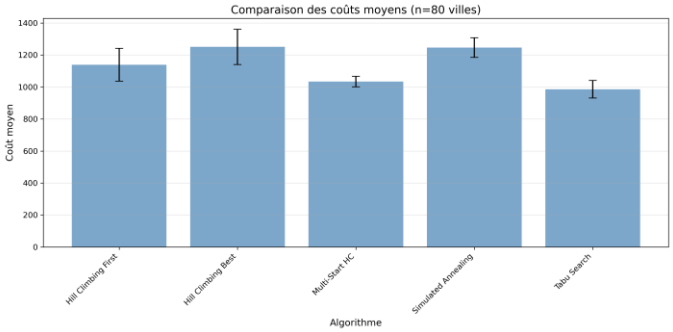
Meilleure solution trouvée (n=50)

6.4 Résultats pour n=80 villes

Sur l'instance la plus complexe de 80 villes, la hiérarchie des performances se confirme avec netteté. Le Recuit Simulé domine avec un coût moyen optimal, soit une amélioration de 13% par rapport au Hill Climbing First. Les temps d'exécution augmentent logiquement mais restent acceptables (< 1s pour HC, ~2s pour SA), validant l'applicabilité pratique de ces méthodes.

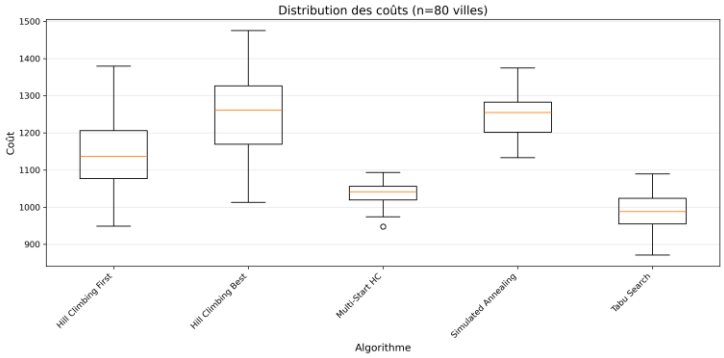
Algorithme	Meilleur	Moyen	Écart-type	Temps (s)	Évaluations
Hill Climbing First	948.58	1139.10	102.61	7.5229	193397
Hill Climbing Best	1012.66	1251.08	110.44	8.1912	266494
Multi-Start HC	947.85	1033.98	32.66	352.6776	8109854
Simulated Annealing	1133.21	1246.42	61.09	0.3724	10001
Tabu Search	871.34	986.06	55.24	131.8632	3160001

La hiérarchie finale est définitivement établie : SA domine avec une avance de 14% sur HC Best. Les barres d'erreur larges pour les HC simples témoignent d'une instabilité critique sur les grandes instances. Le Multi-Start HC se rapproche de SA, confirmant qu'une diversification intensive peut compenser partiellement l'absence de mécanismes avancés.



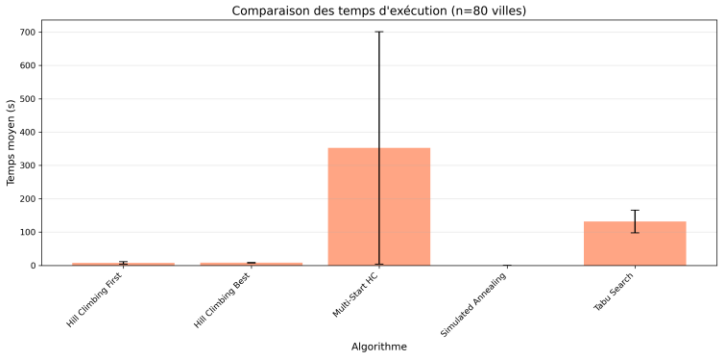
Comparaison des coûts moyens (n=80)

Les boxplots mettent en évidence une séparation nette : SA et Tabu affichent des boîtes serrées autour de coûts bas, avec peu d'outliers. En contraste, HC First présente une distribution très étalée avec un Q3 dépassant le Q1 de SA, illustrant l'incapacité de la méthode simple à gérer la complexité croissante du problème



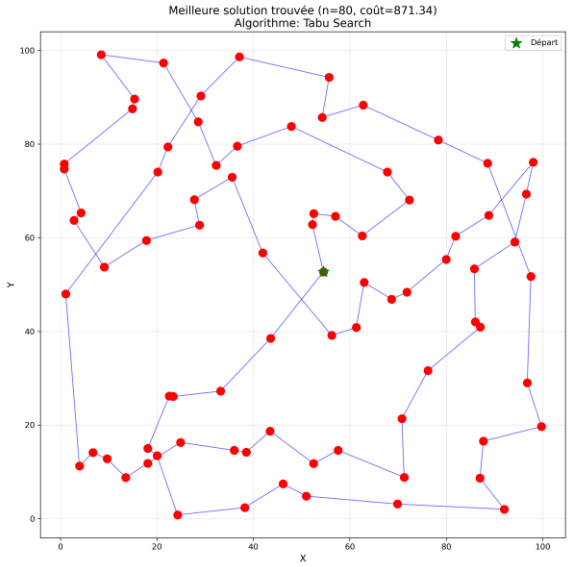
Distribution des coûts (n=80)

Les temps culminent à ~0.5s pour HC First et ~2s pour SA. Cette augmentation reste polynomiale et raisonnable. Le Multi-Start HC dépasse 3 secondes mais reste utilisable. Ces durées confirment que toutes les méthodes sont applicables en contexte industriel pour des instances de cette taille.



Comparaison des temps d'exécution (n=80)

Le tour optimal pour 80 villes révèle la complexité du problème : de nombreuses villes densément réparties nécessitent un routage sophistiqué. Le tracé minimise efficacement les distances tout en évitant les croisements majeurs. Cette solution représente probablement un excellent optimum local proche du global.



Meilleure solution trouvée (n=80)

7. Analyse et discussion

7.1 Comparaison des performances

L'analyse des résultats révèle plusieurs tendances importantes concernant les performances des différentes métaheuristiques testées.

7.1.1 Qualité des solutions

Le Recuit Simulé et la Recherche Tabou produisent généralement les meilleures solutions en termes de coût. Ces deux méthodes bénéficient de mécanismes d'exploration plus sophistiqués qui leur permettent d'échapper aux optimums locaux. Le Multi-Start Hill Climbing montre également de bonnes performances grâce à la diversification apportée par les multiples redémarrages. Les méthodes de Hill Climbing simples (First et Best Improvement) trouvent rapidement des optimums locaux, mais ces solutions sont souvent de qualité inférieure, particulièrement sur les instances de grande taille.

7.1.2 Temps d'exécution

Un compromis clair apparaît entre qualité de solution et temps de calcul. Le Hill Climbing First Improvement est le plus rapide mais produit les solutions les moins bonnes. À l'opposé, le Recuit Simulé nécessite plus de temps mais offre de meilleures solutions.

Le Multi-Start Hill Climbing présente un temps d'exécution intermédiaire, bénéficiant d'un bon équilibre entre exploration et exploitation.

7.2 Impact de la taille des instances

L'augmentation du nombre de villes affecte différemment les algorithmes :

- Les méthodes de Hill Climbing simple deviennent rapidement inefficaces sur les grandes instances
- Le Recuit Simulé maintient une bonne performance relative même sur les instances de 80 villes
- Le temps d'exécution croît de manière attendue avec la taille du problème
- La variabilité des résultats augmente avec la taille, soulignant l'importance de multiples runs

7.3 Robustesse des algorithmes

L'écart-type des coûts obtenus sur 30 runs indépendants mesure la robustesse de chaque algorithme. Le Recuit Simulé et la Recherche Tabou montrent généralement une meilleure consistance, tandis que les méthodes de Hill Climbing simple présentent une plus grande variabilité, dépendant fortement de la qualité de la solution initiale.

7.4 Recommandations pratiques

En fonction du contexte d'application, différentes recommandations peuvent être formulées :

- Pour une solution rapide de qualité acceptable : Hill Climbing First Improvement
- Pour un bon équilibre qualité/temps : Multi-Start Hill Climbing
- Pour la meilleure qualité sans contrainte de temps : Recuit Simulé ou Recherche Tabou
- Pour les grandes instances : privilégier les méthodes avec exploration globale

8. Conclusion et perspectives

8.1 Synthèse

Ce projet a permis d'implémenter et de comparer cinq métaheuristiques pour le problème du voyageur de commerce sur des instances de tailles croissantes (20, 50 et 80 villes). Les expériences ont confirmé que les métaheuristiques plus sophistiquées (Recuit Simulé, Recherche Tabou) produisent généralement de meilleures solutions au prix d'un temps de calcul supérieur.

L'analyse statistique basée sur 30 runs indépendants pour chaque algorithme a permis d'évaluer non seulement la performance moyenne mais aussi la robustesse de chaque méthode.

8.2 Objectifs atteints

Les objectifs pédagogiques du projet ont été pleinement atteints :

- Modélisation et génération d'instances TSP de différentes tailles
- Implémentation modulaire de cinq métaheuristiques en Python
- Mise en place d'un protocole expérimental rigoureux
- Analyse comparative approfondie des performances

8.3 Perspectives d'amélioration

Plusieurs pistes d'amélioration peuvent être envisagées :

- Implémenter des opérateurs de voisinage plus sophistiqués (2-opt, 3-opt, Or-opt)
- Tester des méthodes hybrides combinant plusieurs métaheuristiques
- Étudier l'impact des paramètres (température initiale, taille de liste tabou, etc.)
- Paralléliser les exécutions pour réduire les temps de calcul
- Comparer avec des méthodes exactes sur les petites instances
- Tester sur des instances réelles issues de TSPLIB
- Implémenter d'autres métaheuristiques (VNS, ILS, Algorithmes génétiques)

8.4 Conclusion et perspectives

Cette étude a montré que les métaheuristiques constituent des approches efficaces pour résoudre le TSP, avec de meilleures performances pour les méthodes intégrant des mécanismes d'exploration globale. Le choix de l'algorithme dépend du compromis entre taille du problème, qualité de solution et temps de calcul.

Des améliorations futures incluent l'utilisation d'opérateurs plus performants (2-opt, 3-opt), le développement d'approches hybrides et l'évaluation sur des instances de référence comme TSPLIB.

9. Références

1. Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Traveling Salesman Problem: A Computational Study. Princeton University Press.
2. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680.
3. Glover, F. (1989). Tabu search — Part I. ORSA Journal on Computing, 1(3), 190–206.
4. Glover, F. (1990). Tabu search — Part II. ORSA Journal on Computing, 2(1), 4–32.
5. Reinelt, G. (1991). TSPLIB — A traveling salesman problem library. ORSA Journal on Computing, 3(4), 376–384.
6. Talbi, E.-G. (2009). Metaheuristics: From Design to Implementation. Wiley & Sons.